Implement a character-level language model using RNN

To implement a character-level language model using a SimpleRNN and demonstrate its ability to learn and generate character sequences.

Algorithm
1. Import NumPy and TensorFlow libraries.
2. Define a small text corpus inside the program.
3. Extract all unique characters from the corpus.
4. Create character-to-index and index-to-character dictionaries.
5. Convert the complete text into numerical character sequences.
6. Divide the text into fixed-length input sequences.
7. Assign the next character as the target output.
8. Create an Embedding layer to represent characters.
9. Add a SimpleRNN layer to learn character relationships.
10. Add a Dense softmax output layer.
11. Compile and train the model using categorical cross-entropy.
12. Generate new characters using a starting seed and display the generated text.

Program

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
tf.keras.utils.set_random_seed(24)
# Text corpus
text = (
"deep learning uses neural networks. "
"recurrent neural networks learn sequences. "
) * 20
# Character mapping
chars = sorted(set(text))
char_to_id = {
ch: i for i, ch in enumerate(chars)
}

id_to_char = {
i: ch for ch, i in char_to_id.items()
}
encoded = np.array([
char_to_id[ch] for ch in text
])
seq_len = 20
# Prepare training sequences
X = []
y = []
for i in range(len(encoded) - seq_len):
X.append(encoded[i:i + seq_len])
y.append(encoded[i + seq_len])
X = np.array(X)
y = tf.keras.utils.to_categorical(
y,
num_classes=len(chars)
)
# Create RNN model
model = models.Sequential([
layers.Embedding(len(chars), 32),
layers.SimpleRNN(128),
layers.Dense(len(chars), activation='softmax')
])
# Compile
model.compile(
optimizer='adam',
loss='categorical_crossentropy'
)
# Train
model.fit(
X, y,
epochs=12,
batch_size=64,
verbose=0
```

```
)
# Generate text
seed = "deep learning uses"
generated = seed

for _ in range(100):
last = generated[-seq_len:]
ids = [
char_to_id.get(c, 0)
for c in last
]
ids = [0] * (seq_len - len(ids)) + ids
prediction = model.predict(
np.array([ids]),
verbose=0
)[0]
next_id = int(np.argmax(prediction))
generated += id_to_char[next_id]
print("Seed Text:", seed)
print("Generated Text:", generated)
```

```
Output of Program
Seed Text: deep learning uses
Generated Text:
deep learning uses neural networks...

The exact generated characters depend on training.
```

Result

Thus, a character-level language model was implemented using a SimpleRNN. The model learned character patterns and generated new character sequences from the given seed text.